

# Rapport Projet - Programmation répartie

Alexis CERDA DE ALMEIDA VILACA

Logan SAGET

Taïno HERMANN

Emelyne FOUSSE

8 juin 2026

## Table des matières

|          |                                                             |          |
|----------|-------------------------------------------------------------|----------|
| <b>1</b> | <b>Squelette RMI</b>                                        | <b>2</b> |
| <b>2</b> | <b>Questions du projet</b>                                  | <b>4</b> |
| 2.1      | Shématiser le problème : . . . . .                          | 4        |
| 2.2      | Que fait le client lors d'un appel à un service ? . . . . . | 5        |

# 1 Squelette RMI

## Annuaire

Commande rmiregistry 1099

```
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.RemoteException;

public class Annuaire {
    public static void main(String[] args) {
        String adresse = args[0];
        Registry reg;
        try {
            reg = LocateRegistry.getRegistry(adresse);
            for (String elt : reg.list()) {
                System.out.println(elt);
            }
        } catch (RemoteException e) { // lancer par getRegistry
            System.out.println(e.getMessage());
        }
    }
}
```

## Service

```
import java.rmi.RemoteException;

public class Service implements ServiceInterface {
    public int Methode() throws RemoteException {
        // le throws est obligatoire pour les methodes d une interface RMI
        String host = ""; // ligne qui permet d'afficher l'IP du client !
        try {
            host = RemoteServer.getClientHost();
            System.out.println(host);
        } catch (ServerNotActiveException e) {} // jusqu'ici
        return 0;
    }
}
```

## LancerService

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.server.ServerNotActiveException;

public class LancerService {
    public static void main(String args[]) throws RemoteException,
        ServerNotActiveException {
        try {
            Service nom = new Service(); // creation d un objet (new)
            ServiceInterface rd = (ServiceInterface) UnicastRemoteObject.
                exportObject(nom, 0); // 0 pour que OS donne le port
                automatiquement
            Registry reg = LocateRegistry.getRegistry(1099); // 1099 port
                par defaut des annuaire (modifiable)
            reg.rebind("LeNom", rd); // donner un nom dans l annuaire pour
                le service
        } catch (RemoteException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

## ServiceInterface

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface ServiceInterface extends Remote {
    public int Methode() throws RemoteException; // toutes les
        exceptions ...
}
```

Elle doit être commune entre le Service et le Client

## Client

### Appel

```
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.RemoteException;
import java.rmi.NotBoundException;

public class Appel {
    public static void main(String[] args) {
        String adresse = args[0];
        Registry reg;
        try {
            reg = LocateRegistry.getRegistry(adresse);
            ServiceInterface Objet = (ServiceInterface) reg.lookup("LeNom"
);
            int num = Objet.Methode();
        } catch (NotBoundException e) {
            System.out.println("Reference non trouvee dans l'annuaire");
        } catch (RemoteException e2) {
            System.out.println("Reference(s) non cree(s)");
        }
    }
}
```

## 2 Questions du projet

### 2.1 Shématiser le problème :

En utilisant votre meilleur outil, votre imagination, décrivez et illustrez comment cela pourrait être réalisé, sans rentrer dans les détails JAVA, que vous n'allez pas tarder à mettre en œuvre.

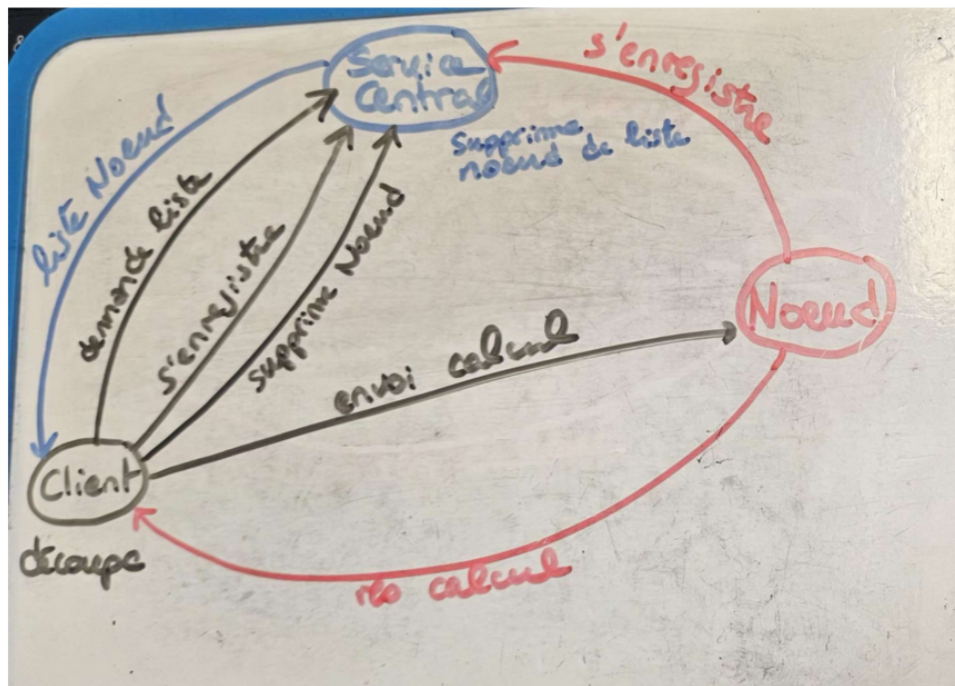


FIGURE 1 – Schéma final de l'architecture RMI

## 2.2 Que fait le client lors d'un appel à un service ?

Tester le programme en modifiant ses paramètres (sur la ligne de commande).

Observer le temps de d'exécution en fonction de la taille de l'image calculée. Vous pouvez faire une courbe (temps de calcul et tailles d'image).

Itération 1 Largeur = 512 Hauteur = 512 Temps de calcul :3182 ms  
 Itération 2 Largeur = 1024 Hauteur = 1024 Temps de calcul :7372 ms  
 Itération 3 Largeur = 1536 Hauteur = 1536 Temps de calcul :13642 ms  
 Itération 4 Largeur = 2048 Hauteur = 2048 Temps de calcul :23977 ms  
 Itération 5 Largeur = 2560 Hauteur = 2560 Temps de calcul :32773 ms  
 Itération 6 Largeur = 3072 Hauteur = 3072 Temps de calcul :48813 ms  
 Itération 7 Largeur = 3584 Hauteur = 3584 Temps de calcul :64370 ms  
 Itération 8 Largeur = 4096 Hauteur = 4096 Temps de calcul :84049 ms  
 Itération 9 Largeur = 4608 Hauteur = 4608 Temps de calcul :107256 ms  
 Itération 10 Largeur = 5120 Hauteur = 5120 Temps de calcul :128906 ms

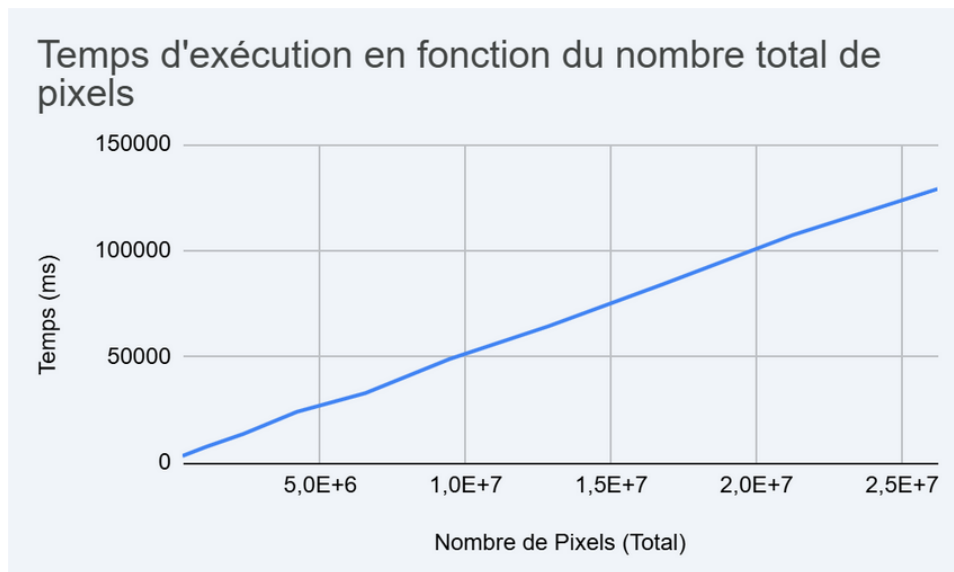


FIGURE 2 – Temps d'exécution en fonction des pixels

En ne modifiant que le fichier `LancerRaytracer.java`, reproduire l'image suivante :

```
int x0 = 0, y0 = 0;
int l = largeur, h = hauteur;

// Chronométrage du temps de calcul
Instant debut = Instant.now();
System.out.println("Calcul de l'image :\n - Coordonnées : "+x0+", "+y0
+" \n - Taille "+ largeur + "x" + hauteur);
Image image = scene.compute(x0, y0, l/2, h/2);
Image image1 = scene.compute(l/2, h/2, l/2, h/2);
Instant fin = Instant.now();

long duree = Duration.between(debut, fin).toMillis();

System.out.println("Image calculée en :"+duree+" ms");

// Affichage de l'image calculée
disp.setImage(image, x0, y0);
disp.setImage(image1, l/2, h/2);
```

Faire un petit schéma de cette architecture en identifiant les choses suivantes :

Le/les processus fixes (ceux qui écoutent sur un port choisi) et les

### **processus mobiles ? (ceux qui rentrent et sortent a leur guise) ?**

Les processus mobiles sont représentés par les différents nœuds de calculs qui peuvent s'enregistrer et se déconnecter. On peut également considérer le client qui est à l'origine de la demande de calcul comme un processus mobile.

Le processus fixe est le service qui va transmettre le calcul aux différents nœuds de calculs. Le service est enregistré sur un port

### **Les types des données échangées entre les processus**

Le client appelle les nœuds inscrits dans le service central. Il envoie le fichier ".txt" et les coordonnées relatives à la création de l'image (deux pour le point initial et deux pour la longueur et la largeur des pixels). Il reçoit un objet "Image" une fois celle-ci construite.

Une fois ces calculs réalisés, les nœuds retournent le résultat au client.

### **Si on veut que les calculs se fassent en parallèle que faut-il faire ?**

Il faut séparer chaque envoi aux nœuds dans des nouveaux Thread.